

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «МУЗЕЙ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Заблоцького Івана Юрійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

.....	стор.
ВСТУП.....	3
ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП	5
1.2 Опис об'єктної частини	6
ПРАКТИЧНА ЧАСТИНА	7
2.2 Опис класі	7
3. ІНТЕРФЕЙС	10
ВИСНОВОК.....	12
СПИСОК ЛІТЕРАТУРИ.....	13

ВСТУП

Метою роботи є створення ієрархії класів на тему «Музей» та застосовування її для розв'язання задачі, визначеної у варіанті. Під час виконання проєкту необхідно застосовувати основні принципи роботи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Відповідно до завдання потрібно реалізувати систему пов'язаних класів, що описують структуру та роботу музею.

Потрібно створити конструктори, поля та методи для роботи з об'єктами. Поля повинні бути закритими, а доступ до них має здійснюватися через відповідні методи. У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції.

ЗАВДАННЯ

Варіант 8б. Створення ієрархії класів на тему «Музей».

У роботі необхідно реалізувати класи, що описують структуру музею, експонати та кімнати. Передбачити можливість роботи різними категоріями експонатів, зберігання інформації про їх назву, розмір, автора, рік, та інші параметри. Також необхідно реалізувати взаємодію між класами, перевірку коректності даних та виконання основних операцій, пов'язаних із функціонуванням музею.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування – це метод програмування, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу, а класи є членами певної ієрархії наслідування [1]. Програмісти спочатку пишуть клас, а на його основі при виконанні програми створюються конкретні об'єкти (екземпляри класів). На основі класів можна створювати нові, які розширюють базовий клас і таким чином створюється ієрархія класів.

Кожний стиль програмування має свою концептуальну базу. Для ОО стилю такою базою є об'єктна модель. Її побудова основана на використанні 3-х основних концепцій[2,3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – механізм, який поєднує дані та методи, що обробляють ці дані і захищає і те і інше від зовнішнього впливу або не вірного використання. Коли і методи і данні об'єднуються таким чином – створюється об'єкт [1].

Наслідування – процес, завдяки якому один об'єкт може придбати властивості іншого, тобто наслідувати властивість іншого об'єкту і додавати риси характерні тільки для нього самого [1].

Поліморфізм – властивість, яка дозволяє одне і те саме ім'я використовувати для вирішення декількох технічно різних задач, тобто основною метою поліморфізму є використання одного імені для задання загальних класу дій. На практиці це значить спроможність об'єктів вибирати внутрішній метод або процедуру, виходячи із типу даних, прийнятих в повідомленні [1].

1.2 Опис об'єктної частини

Програма реалізує консольний застосунок для управління колекцією музейних експонатів. Система підтримує чотири категорії експонатів: картини, скульптури, археологічні знахідки та наукові інструменти. Кожен тип експонатів має власний набір характеристик, але всі вони успадковують спільні властивості від базового класу Exhibit.

Музей складається з кімнат (залів), які характеризуються шириною, довжиною, висотою та корисною площею стін. Кожна кімната містить список своїх експонатів. Об'ємні експонати (скульптури, археологічні знахідки, наукові інструменти) можуть бути розміщені в залі лише в тому випадку, якщо їхній об'єм та габарити не перевищують доступний простір кімнати.

Користувач взаємодіє із системою через консольне меню, де може переглядати наявні експонати, додавати нові до певного залу, а також додавати нові кімнати до музею. При введенні некоректних даних програма виводить відповідне повідомлення про помилку та пропонує спробувати ще раз.

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класі

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

1) Базовий клас «Exhibit» - опис в 2 словах

- a) Приватні поля: name, author, country, year.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Методи:
 - Властивості: Name, Author, Country, Year.
 - ShowYear(int currentYear) – повертає рядок із роком у форматі «р. н.е.» або «р. до н.е.».

2) Клас «VolumeExhibit», нащадок класу «Exhibit»

- a) Приватні поля: width, length, height.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Методи:
 - Властивості: Width, Length, Height (не можуть бути від'ємними).
 - Перевизначення ToString – повертає рядок з описом об'ємного експоната.
 - FitsInRoom(Room room) – перевіряє, чи вміщається експонат за габаритами у задану кімнату.

3) Клас «Picture», нащадок класу «Exhibit»

- a) Приватні поля: height, width.
- b) Конструктори:
 - Конструктор без параметрів

- Конструктор з параметрами
- с) Методи:
 - Властивості: Height, Width (не можуть бути від'ємними).
 - Перевизначення ToString – повертає опис картини.
 - FitsInRoom(Room room) – перевіряє, чи підходить розмір картини для розміщення на стіні кімнати.
- 4) Клас «Sculpture», нащадок класу «VolumeExhibit»
 - а) Приватні поля: успадковані від VolumeExhibit: width, length, height.
 - б) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - с) Методи:
 - Перевизначення ToString – повертає рядок з описом скульптури.
- 5) Клас «ArchaeologicalExhibit», нащадок класу «VolumeExhibit»
 - а) Приватні поля: успадковані від VolumeExhibit: width, length, height.
 - б) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - с) Методи:
 - Перевизначення ToString – повертає рядок з описом археологічного експоната з підтримкою від'ємних років (до н.е.).
- 6) Клас «ScientificInstrument», нащадок класу «VolumeExhibit»
 - а) Приватні поля: успадковані від VolumeExhibit: width, length, height (тип double для точних вимірів).
 - б) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - с) Методи:
 - Перевизначення ToString – повертає рядок з описом наукового інструменту.

7) Клас «Room»

a) Приватні поля: width, length, height, usefulWallArea, volume.

b) Конструктори:

- Конструктор без параметрів
- Конструктор з параметрами

c) Лісти:

- ExhibitList – публічна властивість типу List<Exhibit> для збереження переліку експонатів кімнати.

d) Методи:

- Властивості: Width, Length, Height, UsefulWallArea.
- CanAccommodate(double L, double W, double H) – перевіряє наявність вільного об'єму та зменшує його після успішного розміщення.
- Перевизначення ToString – повертає інформацію про кімнату.

2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить вступне вікно (Рис. 2.1)

```
ви прийшли до музею, щоб подивитися на експонати:
1 передивитись експонати
2 додати експонат
3 додати кімнату
4 вийти
```

Рисунок 2.1 – початкове вікно

Користувач потрапляє сюди коли запускає програму, або виходить з музею для вибору іншого параметру. (Рис.2.1).

Якщо користувач натискає «1», він переходить до перегляду експонатів які вже є в музеї (Рис. 2.2)

```
Картина: назва = Мона Ліза, автор = Леонардо да Вінчі, країна = Італія, рік = 1503 р. н.е., висота = 53, ширина = 77
Археологічний експонат: назва = Тиранозавр Рекс, автор = Тиранозавр, країна = США, рік = 65000000 р. до н.е., ширина = 1
2, довжина = 2, висота = 4
Скульптура: назва = Давид, автор = Мікеланджело, країна = Італія, рік = 1504 р. н.е., ширина = 5, довжина = 2, висота =
1
Науковий інструмент: назва = Телескоп Ганса Янссена, автор = Ганс Янссен, країна = Нідерланди, рік = 1608 р. н.е., ширин
а = 0,5, довжина = 0,15, висота = 0,2
```

Рисунок 2.2 – наявні експонати

У випадку натискання «2», користувачу буде надана можливість додати експонат (Рис. 2.3) для успішного додання експоната користувачу потрібно ввести всі параметри які запрошує програма.

```
Введіть тип експонату (1-картина, 2-скульптура, 3-археологічний експонат, 4-науковий інструмент):
1
Введіть автора експонату:
Да Вінчі
Введіть країну експонату:
Італія
Введіть рік створення експонату:
1454
Введіть висоту картини:
122
Введіть ширину картини:
66
Введіть назву картини:
Мона Ліза
```

Рисунок 2.3 – додавання експонату

При натисканні «3», користувач зможе додати нову кімнату ввівши її розміри (Рис. 2.4)

```
Введіть ширину кімнати (ціле число):  
15  
Введіть довжину кімнати (ціле число):  
25  
Введіть висоту кімнати (ціле число):  
10  
Введіть корисну площу стін (ціле число):  
100
```

Рисунок 2.4 – додавання кімнати

У разі натискання «4», користувач виходить з музею (Рис 2.5)

```
ви прийшли до музею, щоб подивитися на експонати:  
1 передивитись експонати  
2 додати експонат  
3 додати кімнату  
4 вийти  
4  
Дякуємо за відвідування музею! До побачення!
```

Рисунок 2.5 – вихід з музею

ВИСНОВОК

У ході виконання курсової роботи було розроблено програму на мові С# з використанням принципів об'єктно-орієнтованого програмування на тему «Музей». У процесі роботи було створено ієрархію класів для представлення експонатів та кімнат, а також реалізовано взаємодію між ними.

Під час розробки програми були використані основні принципи ООП: інкапсуляція, наслідування та поліморфізм. Для кожного класу були створені відповідні поля, властивості, конструктори та методи.

У результаті виконання курсової роботи були закріплені практичні навички програмування на С#, роботи з класами, колекціями, методами, властивостями та консольним інтерфейсом. Розроблена програма демонструє можливість створення повноцінної системи управління музеєм засобами об'єктно-орієнтованого програмування.

СПИСОК ЛІТЕРАТУРИ

1. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026.
– Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oop>
2. W3School, C# OOP (Object-Oriented Programming). Режим доступу:
https://www.w3schools.com/cs/cs_oop.php
3. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктно-орієнтоване програмування мовою C#. [Електронне видання] / Одеса, 2026. –
Режим доступу: <https://oop-practicum-csharp-851f60.gitlab.io/>
4. Репозиторій GitHub на курсовий – Режим доступу:
<https://github.com/vanyazablotski012-sketch/kursach>

Додаток А. Діаграма класів

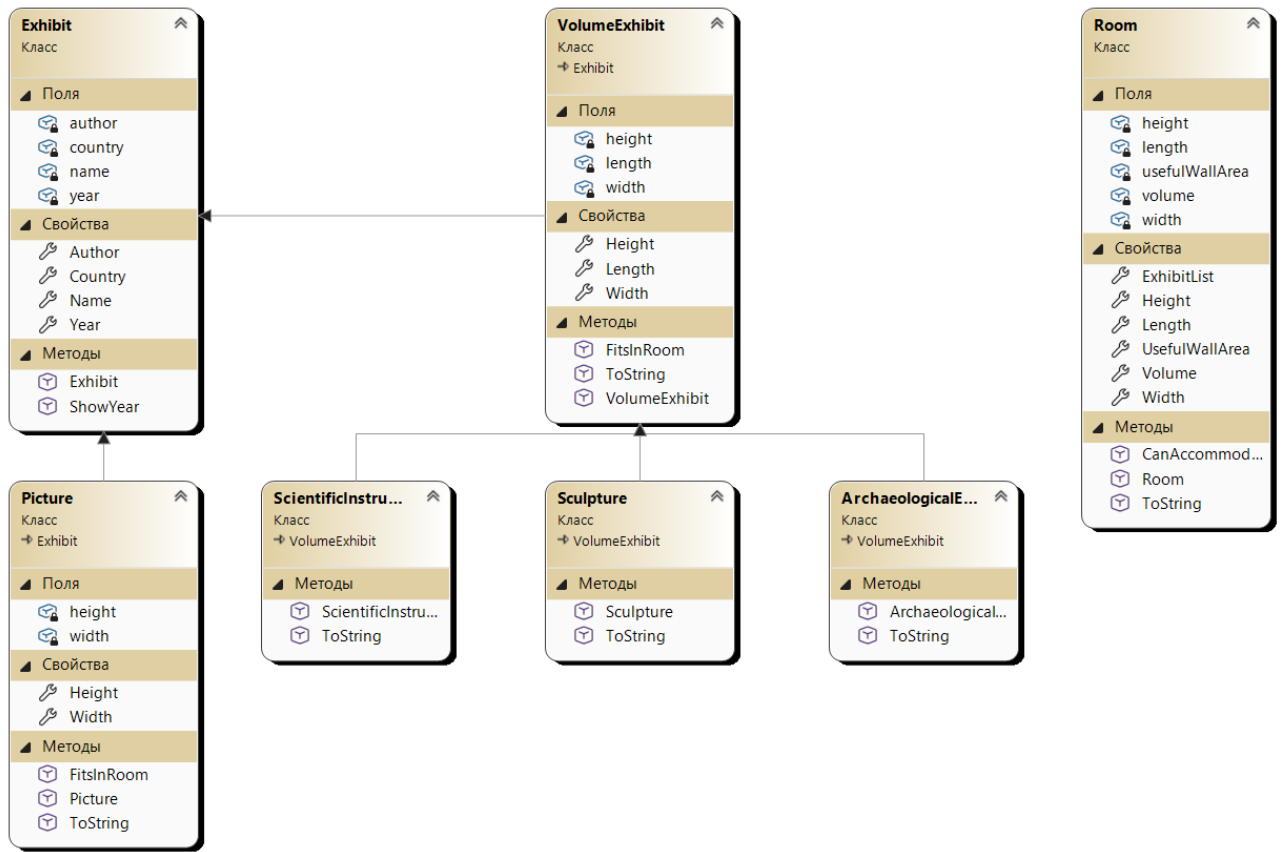


Рисунок А1. – Діаграма класів

Додаток Б. Код класу

```

public class Exhibit
{
    private string name;
    private string author;
    private string country;
    private int year;

    public Exhibit(string name, string author, string country, int year)
    {
        Name = name;
        Author = author;
        Country = country;
        Year = year;
    }

    public string Name
    {
        get { return name; }
        set
        {
            if (string.IsNullOrEmpty(value)) throw new ArgumentException("Назва не
може бути пустою.");
            name = value;
        }
    }
    public string Author
    {
        get { return author; }
        set
        {
            if (string.IsNullOrEmpty(value)) author = "Невідомий";
            author = value;
        }
    }
    public string Country
    {
        get { return country; }
        set
        {
            if (string.IsNullOrEmpty(value)) throw new ArgumentException("Країна
не може бути пустою.");
            country = value;
        }
    }
    public int Year
    {
        get { return year; }
        set
        {
            if (value != 0)
                year = value;
            else
                throw new ArgumentException("Рік не може бути нулем.");
        }
    }

    public string ShowYear(int currentYear)
    {
        if (currentYear > 0)
            return $"{currentYear} р. н.е.";
        else
            return $"{-currentYear} р. до н.е.";
    }
}

```

```

public class Room
{
    private int width;
    private int length;
    private int height;
    private int usefulWallArea;
    private double volume;
    private List<Exhibit> exhibitList = new List<Exhibit>();

    public Room(int width, int length, int height, int usefulWallArea)
    {
        Width = width;
        Length = length;
        Height = height;
        UsefulWallArea = usefulWallArea;
        volume = width * length * height;
    }

    public int Width
    {
        get { return width; }
        set
        {
            if (value < 0) throw new ArgumentException("Ширина не може бути від'ємною.");
            width = value;
        }
    }

    public int Length
    {
        get { return length; }
        set
        {
            if (value < 0) throw new ArgumentException("Довжина не може бути від'ємною.");
            length = value;
        }
    }

    public int Height
    {
        get { return height; }
        set
        {
            if (value < 0) throw new ArgumentException("Висота не може бути від'ємною.");
            height = value;
        }
    }

    public int UsefulWallArea
    {
        get { return usefulWallArea; }
        set
        {
            if (value > Width * Height * 2 + Length * Height * 2) throw new
ArgumentException("Корисна площа стін не може перевищувати загальну площу стін.");
            usefulWallArea = value;
        }
    }

    public List<Exhibit> ExhibitList
    {
        get{ return exhibitList; }
    }

    public double GetVolume()
    {
        return (double)Width * Length * Height;
    }

    public void AddExhibit(Exhibit exhibit)

```



```

    {
        if (exhibit is VolumeExhibit ve)
        {
            if (!CanAccommodate(ve.Length, ve.Width, ve.Height))
                throw new InvalidOperationException($"Експонат '{exhibit.Name}' не
поміщається в кімнату.");
            ExhibitList.Add(exhibit);
        }
    }

    public bool CanAccommodate(double L, double W, double H)
    {
        if (volume >= L * W * H
            && (Width >= W) && (Length >= L) && (Height >= H))
        {
            volume -= L * W * H;
            return true;
        }
        return false;
    }

    public override string ToString()
    {
        return $"Кімната: ширина = {Width}, довжина = {Length}, висота = {Height}, корисна
площа стін = {UsefulWallArea}";
    }
}

```

Лістинг Б.2 – клас Room, аркуш 2

```

namespace kursach.classes
{
    public class ArchaeologicalExhibit : VolumeExhibit
    {
        public ArchaeologicalExhibit(string name, string author, string country, int year,
int width, int length, int height) : base(name, author, country, year, width, length,
height)
        {
        }
        public override string ToString()
        {
            return $"Археологічний експонат: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
        }
    }
}

```

Лістинг Б.3 – клас ArchaeologicalExhibit

```

namespace kursach.classes
{
    public class Picture : Exhibit
    {
        private int height;
        private int width;

        public Picture(string name, string author, string country, int year, int height,
int width) : base(name, author, country, year)
        {
            Height = height;
            Width = width;
        }
    }
}

```

```

public int Height
{
    get { return height; }
    set {
        if (value < 0) {
            throw new ArgumentException("Висота не може бути від'ємною.");
        }
        height = value;
    }
}

```

Лістинг Б.4 – клас Picture, аркуш 1

```

}
public int Width
{
    get { return width; }
    set {
        if (value < 0) {
            throw new ArgumentException("Ширина не може бути від'ємною.");
        }
        width = value;
    }
}
public override string ToString()
{
    return $"Картина: назва = {Name}, автор = {Author}, країна = {Country}, рік = {ShowYear(Year)}, висота = {Height}, ширина = {Width}";
}
public bool FitsInRoom(Room room)
{
    return Width <= room.Width &&
        Height <= room.Height;
}
}
}

```

Лістинг Б.4 – клас Picture, аркуш 2

```

namespace kursach.classes
{
    public class VolumeExhibit : Exhibit
    {
        private double width;
        private double length;
        private double height;
        public VolumeExhibit(string name, string author, string country, int year, double width, double length, double height) : base(name, author, country, year)
        {
            Width = width;
            Length = length;
            Height = height;
        }
        public double Width
        {
            get { return width; }
            set {
                if (value < 0) throw new ArgumentException("Ширина не може бути від'ємною.");
                width = value;
            }
        }
        public double Length
        {
            get { return length; }
            set {

```

```

        if (value < 0) throw new ArgumentException("Довжина не може бути
від'ємною.");
        length = value;
    }
}
public double Height
{
    get { return height; }
    set
    {
        if (value < 0) throw new ArgumentException("Висота не може бути
від'ємною.");
        height = value;
    }
}

```

Лістинг Б.5 – клас VolumeExhibit, аркуш 1

```

    }
}
public override string ToString()
{
    return $"Об'ємний експонат: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
}
public bool FitsInRoom(Room room)
{
    return Width <= room.Width &&
        Length <= room.Length &&
        Height <= room.Height;
}
}
}

```

Лістинг Б.5 – клас VolumeExhibit, аркуш 2

```

namespace kursach.classes
{
    public class Sculpture : VolumeExhibit
    {
        public Sculpture(string name, string author, string country, int year, int width,
int length, int height) : base(name, author, country, year, width, length, height)
        {
        }
        public override string ToString()
        {
            return $"Скульптура: назва = {Name}, автор = {Author}, країна = {Country}, рік
= {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота = {Height}";
        }
    }
}

```

Лістинг Б.6 – клас Sculpture

```

namespace kursach.classes
{
    public class ScientificInstrument : VolumeExhibit
    {
        public ScientificInstrument(string name, string author, string country, int year,
double width, double length, double height) : base(name, author, country, year, width,
length, height)
        {
        }
        public override string ToString()
        {
            return $"Науковий інструмент: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
        }
    }
}

```

```
}  
}  
}
```

Лістинг Б.7 – клас `ScientificInstrument`